

Interfaces de comunicación en sistemas embebidos

Comunicación serial: SPI

Juan José Londoño, M.Sc.

Introducción

SPI (***Serial Peripheral Interface***) es un protocolo de comunicación serial síncrono utilizado para para comunicar circuitos que forman parte del mismo equipo electrónico o que están muy cerca físicamente.

Fue desarrollado por la empresa **Motorola** a finales de los años 80.

En esa época Motorola fabricaba muchos microcontroladores y microprocesadores usados en sistemas embebidos industriales y automotrices.

El objetivo era crear una interfaz que permitiera que el microcontrolador pudiera comunicarse fácilmente con otros circuitos integrados

Introducción

A diferencia de protocolos como I2C o USB, el SPI **no es un estándar formal** regulado por una organización internacional. Esto permite que cada fabricante implemente variaciones en el orden de los bits, los modos de reloj o el número de bits por transferencia, convirtiéndolo más en una familia de interfaces compatibles que en un estándar rígido.

Pese a esta falta de un documento universal único, la arquitectura básica se mantiene constante en todos los dispositivos. Esta flexibilidad es lo que permite una comunicación rápida y eficiente siempre que se ajusten los parámetros específicos de cada periférico.

Introducción

La gran fortaleza de SPI es su capacidad para realizar transferencias de datos a alta velocidad con una lógica de comunicación relativamente simple.

Pero su eficiencia en velocidad suele venir acompañada de un mayor costo en pines y cableado.

Roles en SPI

En SPI existen dos tipos de dispositivos:

Maestro (Master):

Es el dispositivo que inicia la comunicación, genera la señal de reloj, decide con qué dispositivo hablar y dirige el intercambio de información dentro del bus.

Esclavo (Slave):

Son circuitos integrados que están diseñados para responder a comandos enviados por el maestro.

Arquitectura de SPI

Para que los dispositivos puedan intercambiar datos, deben estar conectados entre sí mediante un conjunto de señales eléctricas que forman el bus SPI.

A diferencia de otros protocolos donde muchos dispositivos comparten el mismo canal de comunicación, SPI utiliza una arquitectura en la que el dispositivo principal establece conexiones **directas** con los periféricos.

Todos los dispositivos comparten algunas de las señales del sistema, pero cada periférico tiene **una línea específica** que permite activarlo o desactivarlo.

Arquitectura de SPI

un bus SPI utiliza **cuatro señales** principales:

SCLK – Serial Clock:

Reloj de comunicación del bus SPI. Es generado por el maestro. Cada pulso del reloj marca el momento en que se transmite o se lee un bit de información.

MOSI – Master Out Slave In:

Transporta los datos que salen del dispositivo maestro hacia el esclavo, se usa para enviar información hacia el esclavo.

Arquitectura de SPI

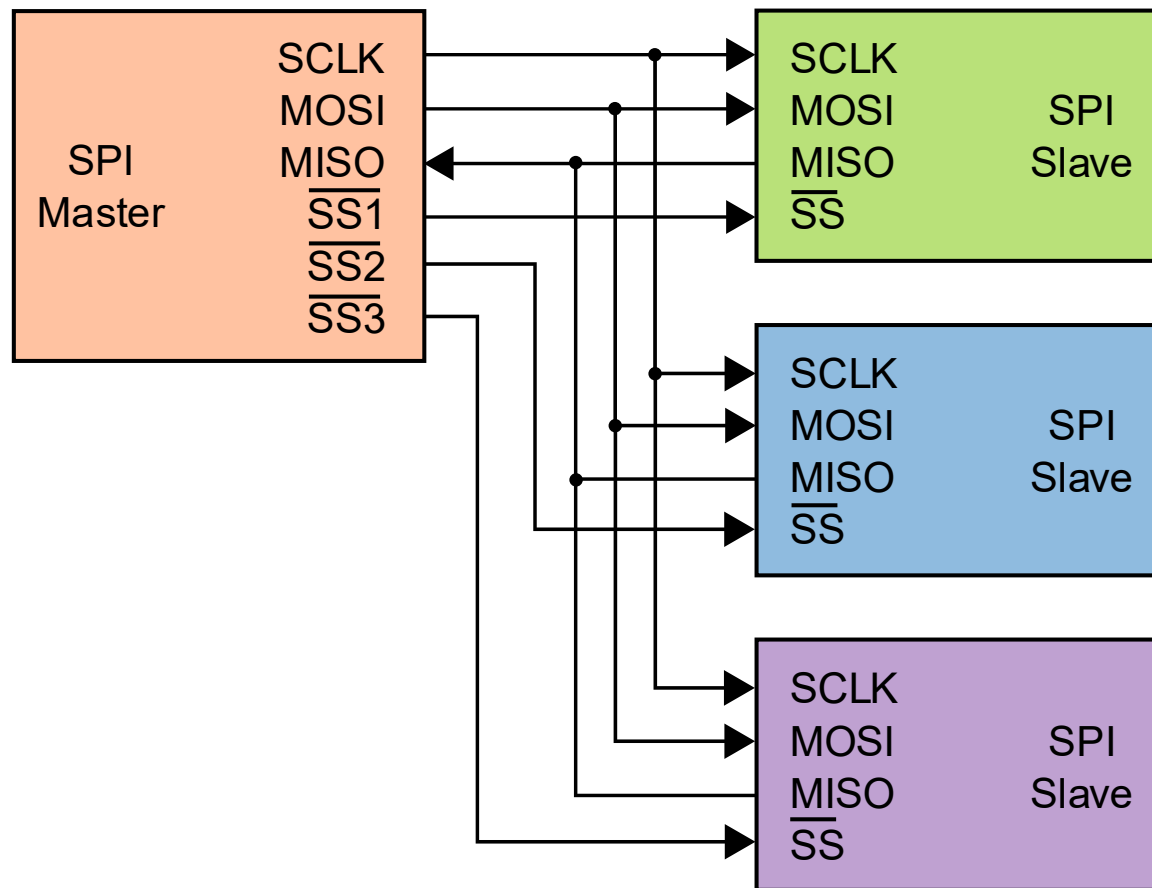
un bus SPI utiliza **cuatro señales** principales:

MISO – Master In Slave Out:

Permite que el esclavo envíe información de regreso hacia el maestro del sistema, se usa para leer información de los esclavos.

CS – Chip Select:

Se utiliza para activar el dispositivo con el que se desea comunicar, normalmente esta señal es **activa en bajo**.



Comunicación sincronizada por CLK

La transferencia de datos ocurre de forma sincronizada con la señal de reloj del bus. Cada pulso del reloj define el momento en el que ocurre la transferencia de un bit. Esto ocurre así:

- El reloj cambia de estado.
- Un bit se coloca en la línea de salida.
- El otro dispositivo lee ese bit.

Esto ocurre repetidamente hasta completar la transmisión del dato.



Comunicación full-duplex

Una característica muy importante de SPI es que la transferencia de datos ocurre en ambas direcciones al mismo tiempo.

Por lo cual, cuando un dispositivo envía un bit, el otro dispositivo también puede enviar un bit **simultáneamente**.

Esto ocurre porque tanto el dispositivo maestro como el esclavo utilizan un mecanismo llamado **registro de desplazamiento**.

Registro de desplazamiento

Un registro de desplazamiento, también llamado ***shift register***, es un circuito que contiene varios bits y que permite desplazar esos bits uno por uno hacia una salida.

Cada pulso de reloj provoca que:

1. Todos los bits del registro se desplacen una posición.
2. El bit más significativo salga hacia la línea de datos.
3. Un nuevo bit entre por el otro extremo.

Registro de desplazamiento

En SPI, ambos dispositivos tienen un registro de desplazamiento conectado al bus.

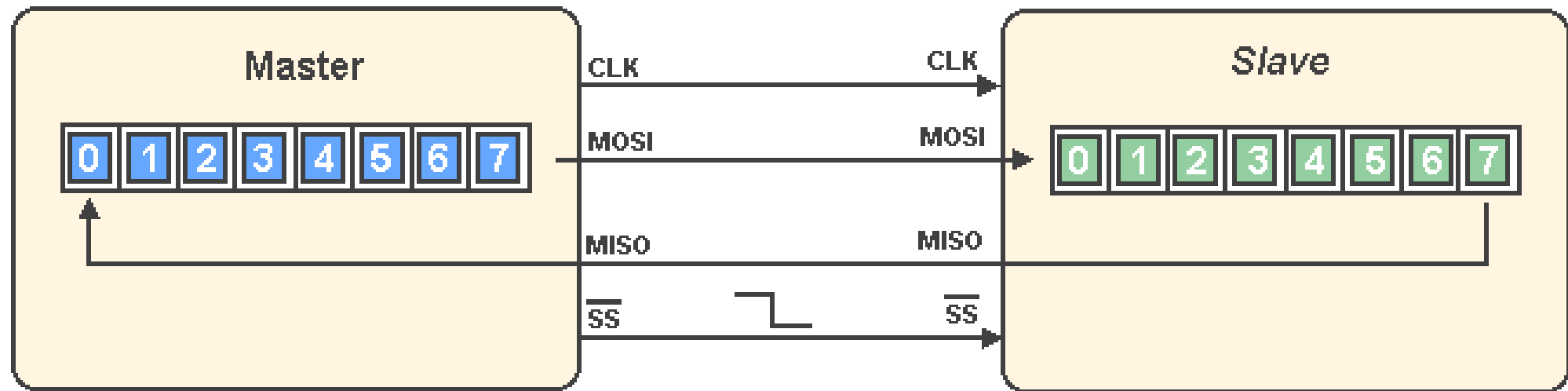
Cuando ocurre un pulso de reloj:

1. El primer dispositivo desplaza un bit hacia el bus.
2. El segundo dispositivo recibe ese bit.
3. Simultáneamente el segundo dispositivo envía otro bit.
4. El primero recibe ese bit.

Registro de desplazamiento

Debido a este mecanismo de registros de desplazamiento, en SPI **siempre que se transmite un dato también se recibe uno**, incluso si el sistema solo quiere leer información de un periférico, el maestro debe enviar datos al mismo tiempo.

Muchas veces se envían simplemente bytes de relleno (*dummy*) para poder generar los pulsos de reloj necesarios (*clockear*), de esta manera se lee la información del esclavo.



Modos de operación de SPI

En SPI los datos se transmiten sincronizados con una señal de reloj, sin embargo, surge una pregunta importante: ¿En qué momento exacto del reloj debe leerse el bit transmitido?

Para describir el comportamiento del reloj SPI se utilizan dos parámetros llamados **CPOL** y **CPHA**, estos parámetros determinan el estado inicial del reloj y en qué momento del ciclo se leen los datos.

Modos de operación de SPI

CPOL (Clock Polarity):

Define el estado del reloj cuando el bus está inactivo.

Existen dos posibilidades:

CPOL = 0: El reloj permanece en nivel bajo cuando no hay comunicación.

CPOL = 1: El reloj permanece en nivel alto cuando no hay comunicación.

Modos de operación de SPI

CPHA (Clock Phase):

Define en qué transición del reloj se capturan los datos.

Existen dos posibilidades:

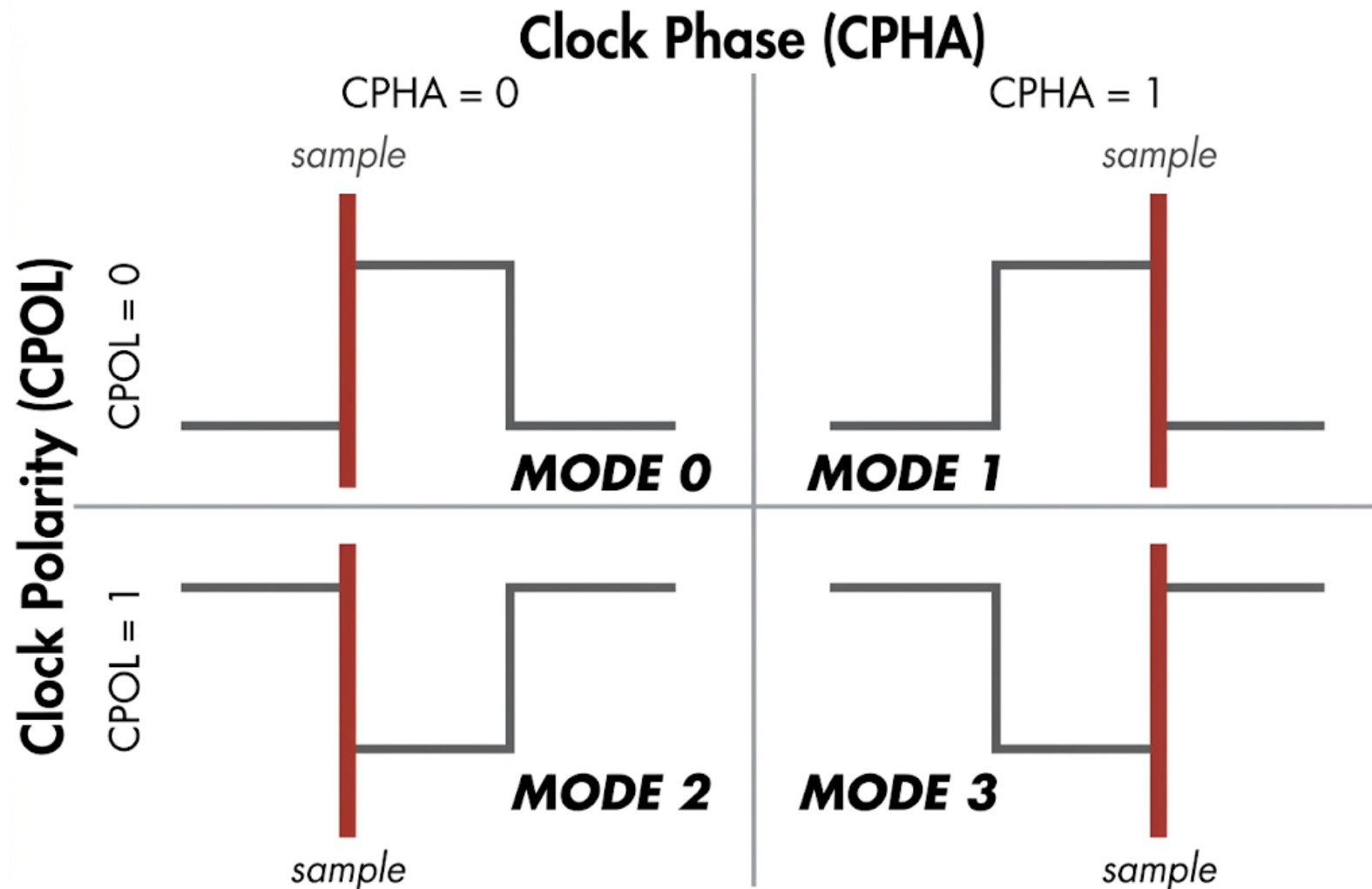
CPHA = 0: El dato se captura en el **primer borde** del reloj.

CPHA = 1: El dato se captura en el **segundo borde** del reloj.

Modos de operación de SPI

Combinando estos dos parámetros se obtienen cuatro modos posibles de SPI.

Modo SPI	CPOL	CPHA
SPI Mode 0	0	0
SPI Mode 1	0	1
SPI Mode 2	1	0
SPI Mode 3	1	1



Velocidad del bus SPI

Uno de los parámetros más importantes es la velocidad del reloj SPI.

La frecuencia del reloj determina qué tan rápido se transmiten los bits en el bus.

Cada pulso del reloj provoca la transferencia de un bit, por lo que la velocidad del reloj define directamente la tasa de transferencia de datos.

Normalmente el dispositivo más lento del bus determina la velocidad máxima permitida.

Orden de los bits

Otro parámetro importante es el orden en el que se transmiten los bits dentro de cada palabra.

Existen dos posibilidades:

MSB first:

Se transmite primero el bit más significativo.

LSB first:

Se transmite primero el bit menos significativo.

SPI en el ESP32

El ESP32 tiene cuatro controladores SPI principales, estos controladores permiten al microcontrolador comunicarse con distintos dispositivos externos usando el protocolo SPI.

Sin embargo, no todos los controladores SPI del ESP32 están disponibles para el usuario de la misma manera.

SPI0 y **SPI1** están utilizados **internamente** por el sistema.

Se usan principalmente para comunicarse con la memoria Flash externa donde está almacenado el firmware.

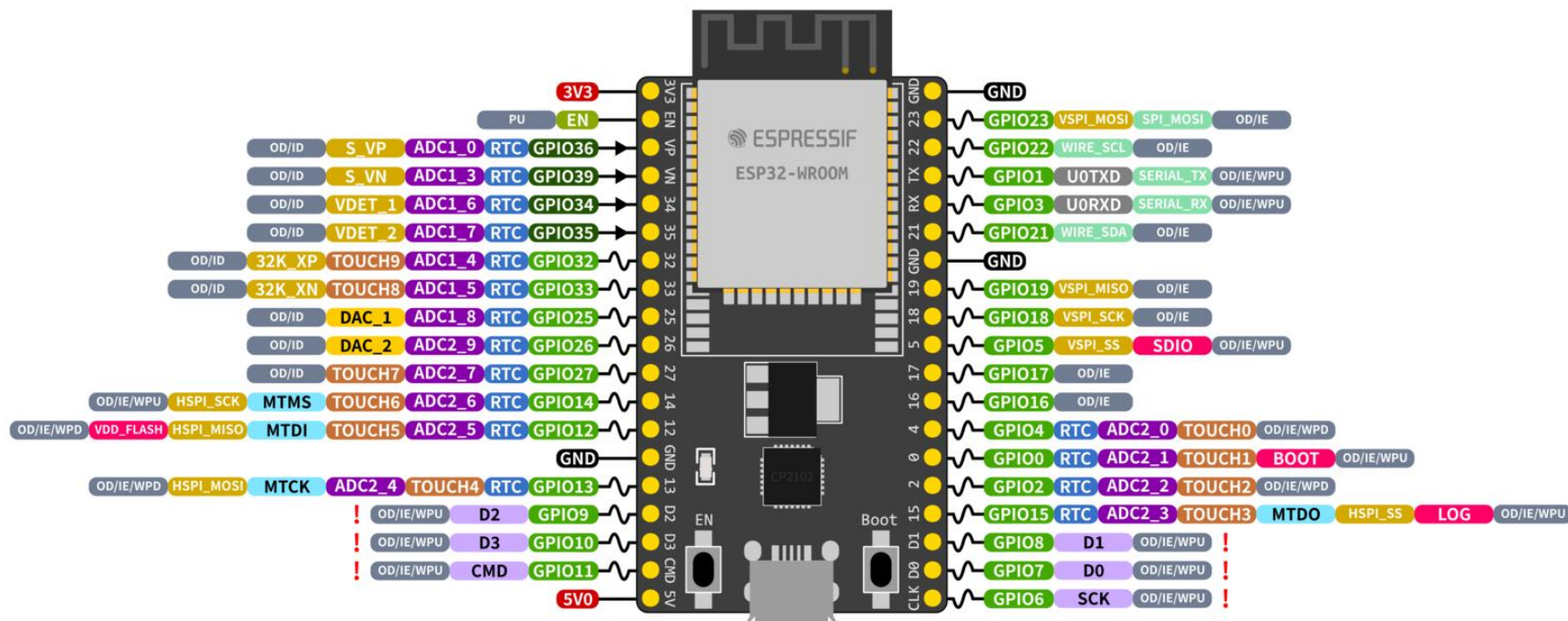
SPI en el ESP32

Los controladores que se utilizan para aplicaciones son:

- SPI2
- SPI3

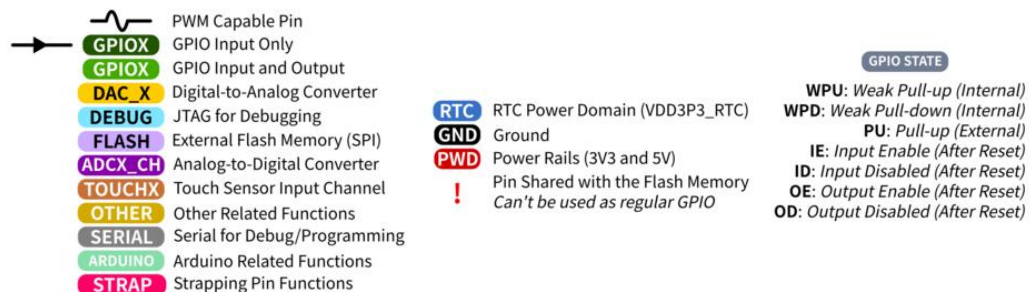
En espidf se llaman **HSPI** y **VSPI**.

Al igual que otros controladores y periféricos del ESP32, las señales SPI no están estrictamente limitadas a pines específicos, y pueden conectarse a casi cualquier pin.



ESP32 Specs

32-bit Xtensa® dual-core @240MHz
 Wi-Fi IEEE 802.11 b/g/n 2.4GHz
 Bluetooth 4.2 BR/EDR and BLE
 520 KB SRAM (16 KB for cache)
 448 KB ROM
 34 GPIOs, 4x SPI, 3x UART, 2x I2C,
 2x I2S, RMT, LED PWM, 1 host SD/eMMC/SDIO,
 1 slave SDIO/SPI, TWAI®, 12-bit ADC, Ethernet



Configuración de SPI



```
#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/spi_master.h"
```

Configuración de SPI



```
#define PIN_NUM_MOSI    23  
#define PIN_NUM_MISO    19  
#define PIN_NUM_CLK     18  
#define PIN_NUM_CS      5
```

Configuración de SPI

```
static spi_device_handle_t spi_dev;

spi_bus_config_t buscfg = {
    .mosi_io_num = PIN_NUM_MOSI,
    .miso_io_num = PIN_NUM_MISO,
    .sclk_io_num = PIN_NUM_CLK,
    .quadwp_io_num = -1,
    .quadhd_io_num = -1,
    .max_transfer_sz = 32
};

spi_device_interface_config_t devcfg = {
    .clock_speed_hz = 1000000, // 1 MHz
    .mode = 0,
    .spics_io_num = PIN_NUM_CS,
    .queue_size = 1
};

spi_bus_initialize(SPI2_HOST, &buscfg, SPI_DMA_DISABLED);
spi_bus_add_device(SPI2_HOST, &devcfg, &spi_dev);
```

Construcción de una transacción SPI

```
spi_transaction_t trans = {  
    .flags = 0,  
    .length = 8,           // bits a transmitir  
    .rxlength = 8,        // bits a recibir  
    .tx_buffer = &tx_data,  
    .rx_buffer = rx_data  
};  
  
spi_device_transmit(spi_dev, &trans);
```